Assignment A: Secure File Vault

Description

In this assignment you will design and build a command-line file vault that protects stored files with password-based authenticated encryption. The cryptographic design is yours to make -- you must choose your algorithms, parameters, and construction, and justify every decision. You will then analyse a deliberately broken vault implementation, identify its flaws, and demonstrate at least one of them in code.

The construction you build -- PBKDF2 key derivation feeding a symmetric cipher with a separate authentication tag -- is the same pattern standardised in PKCS#5 (RFC 8018), used in encrypted PEM private keys, PKCS#12 keystores, and encrypted ZIP archives. The flaws in `vault_broken.py` are not hypothetical: the 2022 LastPass breach exposed encrypted user vaults to offline attack, and their practical crackability depended directly on how many PBKDF2 iterations each account had been configured with.

Learning Objectives

- Design a password-based authenticated encryption scheme from scratch and justify each cryptographic choice.
- Implement PBKDF2 key derivation, AES-CBC encryption, and HMAC-SHA256 authentication correctly and in the right order.
- Identify and exploit weaknesses in a broken implementation: fixed salts, low iteration counts, and MAC-then-encrypt ordering.
- Articulate the concrete attacks that each flaw enables.

Setup

```
pip install pycryptodome
```

`hashlib`, `hmac`, `os`, `json`, and `getpass` are all in the Python standard library.

Part 1 -- Build a Secure Vault

Requirement 1 -- Command-line interface

Your solution must be a single file named `vault.py` that supports three commands:

```
python vault.py add <filename>
python vault.py get <filename>
python vault.py list
```

- `add <filename>`: stores the file securely in the vault.
- `get <filename>`: retrieves and restores the file from the vault.
- `list`: prints the names of all files currently stored in the vault.

The vault stores its data in a directory named `.vault` in the current working directory.

Requirement 2 -- Cryptographic design

Your vault must satisfy all of the following security properties. How you achieve them is your decision:

- The file contents must be encrypted with a symmetric cipher using a key derived from the master password.
- Each stored file must use a fresh random salt so that two files encrypted with the same password produce different key material.
- Separate keys must be used for encryption and authentication -- a single key must not serve both roles.
- The stored blob must be authenticated so that any modification to the ciphertext, IV, or salt is detected b efore decryption.
- A wrong password must be rejected with a clear error message. The vault must not produce garbled output or silently return incorrect data.

You are free to choose any reasonable algorithm, key length, and parameter values -- but you must justify every choice in the write-up.

Requirement 3 -- Tamper detection demonstration

Manually corrupt one byte anywhere in a stored blob file and run `vault.py get`. Show that the vault detects the modification and prints an error. Describe what you did in the write-up.

Part 2 -- Analyse a Broken Vault

The file `vault_broken.py` contains a working, but insecure vault implementation with three deliberate flaws. Study it carefully.

Requirement 4 -- Identify the flaws

Identify all three deliberate security flaws in `vault_broken.py`. For each one, name it, quote the relevant line(s), and explain what attack it enables.

Requirement 5 -- Demonstrate the fixed-salt flaw in code

Write a short script that:

1. Uses `vault_broken.py`'s `derive_keys()` to derive keys for two different filenames using the same password.
2. Prints the derived keys for both and asserts they are identical.
3. Explains in a comment why this is dangerous.

Requirement 6 -- Explain the MAC-then-encrypt flaw

In `vault_broken.py`, the MAC is computed over the plaintext and then encrypted together with it. Explain in 3-4 sentences what attack surface this creates and why your `vault.py` avoids it.

Submission

Submit:

- `vault.py` -- your complete implementation.
- A short analysis script for Requirement 5.
- A written explanation (maximum one A4 page) answering all five questions below.

Write-up questions:

(a) Justify every cryptographic parameter you chose in `vault.py`: symmetric cipher and mode, key length, KDF and iteration count, MAC construction, and blob layout. For the iteration count in particular, explain the tradeoff between security and performance.

(b) Why must separate keys be used for encryption and authentication? What can go wrong when a single key serves both roles?

(c) Why is the authentication tag computed over the ciphertext rather than the plaintext, and why must it cover the salt and IV as well?

(d) What would an attacker be able to do if the salt were stored as a fixed constant rather than generated fresh per file?

(e) Look up Argon2 (RFC 9106), the winner of the 2015 Password Hashing Competition. What does memory-hardness mean, and why does it make Argon2 more resistant to GPU-based attacks than PBKDF2? If you were designing `vault.py` today for production use, what would you change and why?

Grading

Component	Marks
Correct cryptographic implementation (vault.py)	25%
Design justification in write-up question (a)	15%
Correct tamper and wrong-password rejection	10%
Flaw identification -- all three flaws named and explained (Req 4)	20%
Fixed-salt demonstration script (Req 5)	15%
MAC-then-encrypt explanation (Req 6 + write-up c)	15%

Hints

- Use Python's standard `hmac` module: `hmac.new(key, msg, digestmod)`. To avoid a name clash with the module itself, import it as `import hmac as hmac_module` and call `hmac_module.new(...)`.
- PBKDF2 can produce more than 16 bytes of key material in a single call -- one call with a larger `dklen` is preferable to two separate calls.
- To corrupt a blob byte without a hex editor:

```
python -c "d=open('.vault/file.bin','rb').read();
open('.vault/file.bin','wb').write(d[:40]+bytes([d[40]^0xFF])+d[41:])"
```

  Replace `file.bin` with the actual blob filename in `.vault/`.